CyberLabX – Security & Access Document (SAD)

Version: 1.0

Product: CyberLabX

Security Goal: Ensure students can safely learn cybersecurity in an isolated environment while protecting the platform, user data, and hosting infrastructure.

---

1. Security Philosophy

CyberLabX is different from a normal web application.

Students will execute Linux commands, network scans, and cybersecurity tools. That means security is part of the product itself.

Our security model has four priorities:

1. Protect student accounts.
2. Protect college data.
3. Prevent students from escaping the lab environment.
4. Prevent CyberLabX from being used to attack real systems.

---

2. Authentication Strategy

Recommended Authentication

Email + Password + JWT (HttpOnly Cookies)

This is the best balance between security and implementation complexity for an educational SaaS.

Login Flow

```
None
Student

↓

Email + Password

↓

Backend verifies password

↓

JWT created

↓

JWT stored in HttpOnly Cookie

↓

Student is logged in
```

Why HttpOnly Cookies?

Many beginner projects store tokens in localStorage.

We should not.

Reasons:

- JavaScript cannot read HttpOnly cookies.
- Better protection against XSS.
- More secure session handling.
- Easier future enterprise deployment.

Password Policy

Minimum:

- 8 characters
- One uppercase letter
- One lowercase letter
- One number

Recommended:

- 12+ characters

Passwords are never stored directly.

Only password hashes are stored.

Password Storage

Use

```
None
bcrypt
```

or

```
None
Argon2
```

Argon2 is stronger.

bcrypt is perfectly acceptable for Version 1.

---

3. User Roles

CyberLabX has three roles.

Student

Can

- Register/Login
- View available experiments
- Launch labs
- Execute commands inside assigned containers
- View reports
- Download personal reports
- Take quizzes
- View own progress

Cannot

- Access another student's reports
- View another student's commands
- Modify experiments
- Start unlimited lab sessions
- Access Docker
- Access backend APIs directly
- Execute commands outside the container
- Scan public Internet
- Access host machine

Faculty

Can

- Login
- View assigned classes
- Create experiments
- Edit experiment theory
- View student reports
- Assign labs
- View analytics
- Export grades
- Disable experiments

Cannot

- Access Docker host
- Access server terminal
- Modify administrator accounts
- View student passwords
- Delete entire system

Administrator

Can

Everything except database access outside the application.

Can:

- Create faculty
- Create students
- Delete users
- View logs
- Configure experiments
- Manage Docker
- Configure system settings
- View analytics
- Suspend users

Cannot

- View plain text passwords
- Directly access student lab containers without audit logging
- Bypass audit trails

Every admin action should be recorded.

---

4. Role Permissions Matrix

Action	Student	Faculty	Admin
Login	✅	✅	✅
Start Lab	✅	❌	✅
Execute Commands	✅	❌	✅
View Own Reports	✅	✅	✅
View All Reports	❌	✅	✅
Create Experiment	❌	✅	✅
Delete Experiment	❌	❌	✅
Manage Users	❌	❌	✅
Configure Docker	❌	❌	✅

---

5. Database Access Rules

Every database query should verify ownership before returning data.

Examples:

Student Reports

Allowed

```
None
Student ID == Report Owner
```

Denied

```
None
Student ID != Report Owner
```

Student Attempts

Allowed

```
None
attempt.studentId == currentUser.id
```

Faculty

Faculty may view

Only students assigned to them.

Not every student in the system.

Admin

Can view everything.

---

6. Row-Level Security (Plain English)

Think of the database as a giant filing cabinet.

Each user should only be able to open the folders they own.

Students

Can only access:

- Their profile
- Their reports
- Their lab sessions
- Their command history
- Their quiz scores

Faculty

Can access:

- Assigned students
- Assigned reports
- Assigned experiments

Cannot access unrelated departments.

Admin

Can access every record.

---

7. Lab Isolation Security

This is the most important part of CyberLabX.

Each student receives

```
None
One Docker Container
```

Example

```
None
Student A

↓

Container A
```

Student B

↓

Container B

Student C

↓

Container C

No student should ever share a container.

Containers should have

✅ CPU limit

Example

```
None
1 CPU
```

Memory limit

```
None
1GB
```

Read-only filesystem where possible

No privileged mode

No root access to the Docker host

---

8. Network Security

Students should never reach

- Google
- GitHub
- College servers
- Public IPs

Instead

```
Student

↓

Container

↓

Internal Network

↓

Practice Machines
```

Only internal IP addresses.

---

9. Command Restrictions

Students can execute

```
nmap

ping

tcpdump

ls

pwd

cat
```

```
cd
```

Commands like

```
None
shutdown

reboot

rm -rf /

systemctl

docker

mount
```

should either be unavailable or harmless within the isolated container.

The key protection is isolation, not simply blocking commands. Even if a student deletes everything in their container, the platform should be able to destroy it and create a fresh one.

---

10. Rate Limiting

Login

Maximum

```
None
5 attempts

↓
```

```
Lock for 15 minutes
```

API

```
None
100 requests/minute
```

Terminal

```
None
One active lab per student
```

---

11. Audit Logs

Every important action should be recorded.

Store

```
None
User

Action

Timestamp

IP

Result
```

Example

```
None
Faculty
```

```
Exported grades

July 28

Success
```

---

12. Error Handling Guide

Login Failed

Message

```
None
Invalid email or password.
```

Never reveal

```
None
Email does not exist.
```

That prevents account enumeration.

Session Expired

Automatically redirect

```
None
Login

↓
```

```
Continue where you left off
```

Container Failed

Show

```
None
Unable to start the lab.

Please try again.

If the issue continues,
contact your instructor.
```

Never display

```
None
Docker Error 502
```

Database Offline

Student sees

```
None
The platform is temporarily unavailable.

Please try again later.
```

Administrator log

```
None
MongoDB Connection Refused
```

API Failure

Student

```
None
Something went wrong.

Retry
```

Developer logs

```
None
Complete stack trace
```

Terminal Lost Connection

Automatically reconnect.

If reconnection fails

Save progress.

Restart container if needed.

PDF Report Failed

Student

```
None
Report generation failed.
```

```
Please retry.
```

System retries automatically.

---

13. Edge Cases

Student closes browser

When returning

Resume lab.

Internet disconnects

Keep container alive for a limited time (for example, 10–15 minutes).

Reconnect if possible.

Student refreshes page

Reconnect to the existing session instead of creating a new one.

Student opens two tabs

Only one terminal should be active.

The second tab becomes read-only or reconnects to the same session.

Container crashes

Destroy it.

Create a new one.

Restore progress if possible.

Student forgets password

Email reset link.

Expire after

```
None
30 minutes
```

Faculty deletes experiment

Prevent deletion if

Students are currently using it.

Student exits during quiz

Save answers automatically.

Browser crashes

Resume session after login.

Duplicate registration

Display

```
None
Email already registered.
```

Docker image missing

Do not attempt to start the lab.

Notify administrator.

Server reboot

Restore active sessions where possible.

Student runs infinite loop

Automatically terminate after CPU limit exceeded.

Student fills disk

Container storage quota exceeded.

Reset container.

Student launches 100 labs

Reject.

Maximum

```
None
One active lab
```

Student attempts to access host

Log the event.

Terminate container.

Notify administrator if repeated.

Student attempts public network scanning

Block network access outside the lab network.

Record the attempt.

---

14. Monitoring

Track

- Failed logins
- Container failures
- API errors
- Lab startup time
- Session duration
- Active containers
- Memory usage
- CPU usage
- Storage usage

---

15. Security Checklist Before Launch

- HTTPS enabled.
- Secure cookies (`HttpOnly`, `Secure`, `SameSite`).
- Passwords hashed.
- JWT expiration configured.
- Input validation on every endpoint.
- Role-based authorization enforced.
- Docker containers isolated and resource-limited.
- Internal-only lab network.
- Rate limiting enabled.
- Audit logging enabled.
- Regular database backups.
- Secrets stored outside source code.
- Dependency vulnerability scanning completed.

---

16. Incident Response

If suspicious activity is detected:

1. Record the event in the audit log.
2. Suspend the affected lab session.
3. Preserve logs for investigation.
4. Notify the administrator.
5. Allow the student to continue only after the environment has been reset if appropriate.

The goal is to protect the platform while minimizing disruption to legitimate users.

Recommendations Before Version 1

There are three security decisions I would make now because they are much harder to retrofit later:

1. **Treat lab containers as disposable.** Never rely on their state. If something goes wrong, destroy the container and create a fresh one instead of trying to repair it.
2. **Keep the lab network completely isolated from the public Internet.** All Nmap scans and future exercises should target intentionally vulnerable machines inside your private Docker network. This reduces both security risk and legal concerns.
3. **Separate permissions into authentication, authorization, and lab control.** Logging in proves who the user is, role-based access control determines what they are allowed to

do, and the lab engine independently verifies whether they can start, reconnect to, or terminate a lab session. Keeping these concerns separate makes the platform easier to secure and maintain as it grows.